Welcome to Mars. Please don’t die.

Amedeo Benedetti
Marco Cotogni
Leonard Ioan Grigoras
Giuseppe De Massis

Mission Briefing

- Setting the Scene: The year is 2036. After a grueling 12-hour shift at SpaceY, an accidental shipping error has ”promoted” you to Mars Operations while you were sleeping at your desk.
- The Crisis: You wake up in a fragile habitat where the automation stack is partially destroyed and devices speak incompatible dialects.
- Technical Challenges: The environment consists of heterogeneous devices: some stream telemetry asynchronously, while others respond only to manual polling.
- The Objective: Rebuild a distributed automation platform from scratch to ensure survival.

Data Flow

1 Acquisition

2 Normalization

3 Message Bus

4 Rule Evaluation

5 Trigger

6 Actuation

Tech stack

Layer	Technology	Version / Notes
Runtime	Python	3.10+ – all microservices
Web Framework	FastAPI	ASGI – uvicorn server. Pydantic v2 for validation
HTTP Client	httpx	Async (AsyncClient) for SSE streams and REST calls
Message Broker	RabbitMQ	4.2.4-management – topic exchange, durable
AMQP Client	pika	BlockingConnection with retry loop – publisher + consumer
Database	PostgreSQL	16 – Schema initialised via init.sql
DB Driver	psycopg2 / psycopg	Via db_config.py – env-based connection string
ORM / Query	Raw SQL (psycopg2)	RulesRepository uses direct SQL queries, no ORM
Data Validation	Pydantic v2	BaseModel for all messages and rules
Containerisation	Docker + Compose	8 services, internal Docker bridge network
Frontend	Jinja2 + Vanilla JS	No JS framework – fetch API + DOM manipulation
CSS	Custom CSS	Dark theme, CSS custom properties, responsive layout

Ingestion Service – Data Collection Layer

Goal

- Collect sensor data from the Mars IoT simulator
- Handle heterogeneous device interfaces

Responsibilities

- Discover available devices
- Retrieve sensor data
- Normalize heterogeneous payloads
- Publish events to RabbitMQ

Supported sources

- REST sensors (polling)
- Telemetry streams (Server-Sent Events)

Key idea

Event-driven architecture: ingestion produces events that other services consume.

Data Normalization Pipeline

Problem

Devices produce heterogeneous payloads.

Examples:

- Scalar sensors
- Chemistry sensors
- Telemetry topics
- Environmental measurements

Solution:

Convert all payloads into a Unified Event Schema

python.py

```python
class MeasurementEvent(BaseModel):
    event_id: str = Field(..., description="Unique event identifier")
    source_kind: Literal["rest_sensor", "telemetry_topic"] = Field(
        ...,
        description="The source kind of the event"
    )
    source_name: str = Field(
        ...,
        description="The logical name of the sensor or telemetry topic"
    )
    schema_family: Literal[
        "rest.scalar.v1",
        "rest.chemistry.v1",
        "rest.particulate.v1",
        "rest.level.v1",
        "topic.power.v1",
        "topic.environment.v1",
        "topic.thermal_loop.v1",
        "topic.airlock.v1",
    ] = Field(
        ...,
        description="The schema family of the original payload"
    )
    timestamp: datetime = Field(
        default_factory=utc_now,
        description="The timestamp of the event"
    )
    status: Literal["ok", "warning"] = Field(
        ...,
        description="The status of the event"
    )
    measurements: List[Measurement] = Field(
        ...,
        description="The normalized measurements associated with the event"
    )
```

Actuator Service – Execution Layer

Goal
Execute automation actions triggered by rules.

How it works

1. Consume actuator events from RabbitMQ
2. Validate the event payload
3. Send command to simulator actuator API

POST /api/actuators/{actuator_name}

{ "state": "ON" }

Ingestion

Generated Event

python.py

```python
class MeasurementEvent(BaseModel):
    event_id: str = Field(..., description="Unique event identifier")
    source_kind: Literal["rest_sensor", "telemetry_topic"] = Field(
        ...,
        description="The source kind of the event"
    )
    source_name: str = Field(
        ...,
        description="The logical name of the sensor or telemetry topic"
    )
    schema_family: Literal[
        "rest.scalar.v1",
        "rest.chemistry.v1",
        "rest.particulate.v1",
        "rest.level.v1",
        "topic.power.v1",
        "topic.environment.v1",
        "topic.thermal_loop.v1",
        "topic.airlock.v1",
    ] = Field(
        ...,
        description="The schema family of the original payload"
    )
    timestamp: datetime = Field(
        default_factory=utc_now,
        description="The timestamp of the event"
    )
    status: Literal["ok", "warning"] = Field(
        ...,
        description="The status of the event"
    )
    measurements: List[Measurement] = Field(
        ...,
        description="The normalized measurements associated with the event"
    )
```

Normalization

Ingestion

The Ingestion Service polls every 5 s and listens to SSE streams. Payloads are normalised into a Pydantic UnifiedEvent wrapping a MeasurementEvent.

- poller.py
- telemetry_listener.py
- normalizer.py
- simulator_client.py

Supported Schema Families

rest.scalar.v1	REST	metric, value, unit
rest.chemistry.v1	REST	measurements[] (metric/value/unit)
rest.level.v1	REST	level_pct (%), level_liters (L)
rest.particulate.v1	REST	pm1, pm25, pm10 (ug/m3)
topic.power.v1	SSE	voltage (V), current (A), power (kW), cumulative_energy (kWh)
topic.environment.v1	SSE	generic measurements[]
topic.thermal_loop.v1	SSE	temperature (C), flow_rate (L/min)
topic.airlock.v1	SSE	cycles_per_hour, last_state

Message Bus

RabbitMQ

Each normalised event is published to the topic exchange mars.events with routing key depending on the type of event.

Message Bus

RabbitMQ

python.py

```python
class RabbitMQPublisher:
    def __init__(
        self,
        exchange: str = RABBITMQ_EXCHANGE,
    ):

        self.exchange = exchange
        self.connection = None
        self.channel = None



    def connect(self) -> None:

        credentials = pika.PlainCredentials(RABBITMQ_USER, RABBITMQ_PASS)

        parameters = pika.ConnectionParameters(
            host=RABBITMQ_HOST,
            port=RABBITMQ_PORT,
            credentials=credentials,
        )

        while True:
            try:
                self.connection = pika.BlockingConnection(parameters)
                self.channel = self.connection.channel()

                self.channel.exchange_declare(
                    exchange=self.exchange,
                    exchange_type="topic",
                    durable=True,
                )

                logger.info("Connected to RabbitMQ")

                return

            except pika.exceptions.AMQPConnectionError:
                logger.warning("RabbitMQ not ready yet, retrying in 3 seconds...")
                time.sleep(3)


    def publish(self, routing_key: str, payload: dict) -> None:
        if self.channel is None:
            raise RuntimeError("RabbitMQ channel is not connected")
        try:
            self.channel.basic_publish(
                exchange=self.exchange,
                routing_key=routing_key,
                body=json.dumps(payload, default=str).encode("utf-8"),
                properties=pika.BasicProperties(
                    content_type="application/json",
                    delivery_mode=2,
                ),
            )
        except pika.exceptions.ChannelWrongStateError:
            try:
                self.close()
                self.connect()
            except:
                self.connect()

    def close(self) -> None:
        if self.connection and self.connection.is_open:
            self.connection.close()
```

Message Bus

RabbitMQ

python.py

```python
class RabbitMQConsumer:

    def __init__(self, consumer_name: str, exchange: str, routing_keys: list[str], callback_func: Callable) -> None:
        self.name = consumer_name
        self.routing_keys = routing_keys
        self.connection = None
        self.channel = None
        self.exchange = exchange
        self.callback_func = callback_func



    def connect(self) -> None:

        credentials = pika.PlainCredentials(RABBITMQ_USER, RABBITMQ_PASS)

        parameters = pika.ConnectionParameters(
            host=RABBITMQ_HOST,
            port=RABBITMQ_PORT,
            credentials=credentials,
        )

        while True:
            try:
                self.connection = pika.BlockingConnection(parameters)
                self.channel = self.connection.channel()

                self.channel.exchange_declare(
                    exchange=self.exchange,
                    exchange_type="topic",
                    durable=True,
                )

                logger.info("Connected to RabbitMQ")

                self._declare_queues()

                return

            except pika.exceptions.AMQPConnectionError:
                logger.warning("RabbitMQ not ready yet, retrying in 3 seconds...")
                time.sleep(3)
```

Message Bus

RabbitMQ

python.py

```python
def _declare_queues(self) -> None:

        for routing_key in self.routing_keys:

            queue_name = f"{self.name}.queue.{routing_key}.events"

            self.channel.queue_declare(
                queue=queue_name,
                durable=False,
            )

            self.channel.queue_bind(
                exchange=RABBITMQ_EXCHANGE,
                queue=queue_name,
                routing_key=routing_key,
            )

            self.channel.basic_consume(
                queue=queue_name,
                on_message_callback=self._on_message,
            )

            logger.info(
                "Queue created queue=%s bound_to=%s",
                queue_name,
                routing_key,
            )

    def _on_message(self, ch, method, properties, body: bytes) -> None:

        try:

            payload = json.loads(body.decode("utf-8"))

            event = UnifiedEvent.model_validate(payload)

            self.callback_func(event)

            ch.basic_ack(delivery_tag=method.delivery_tag)

        except ValidationError as exc:

            logger.exception("Invalid UnifiedEvent payload: %s", exc)

            ch.basic_nack(
                delivery_tag=method.delivery_tag,
                requeue=False,
            )

        except Exception as exc:

            logger.exception("Unexpected consumer error: %s", exc)

            ch.basic_nack(
                delivery_tag=method.delivery_tag,
                requeue=True,
            )

    def start_consuming(self) -> None:

        if self.channel is None:
            raise RuntimeError("Consumer channel not connected")

        logger.info("Consumer started")

        self.channel.start_consuming()

    def close(self) -> None:

        if self.connection and self.connection.is_open:
            self.connection.close()
```

Message Bus

RabbitMQ

python.py

```
rabbitmq:
    image: rabbitmq:4.2.4-management
    container_name: mars-rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: mars
      RABBITMQ_DEFAULT_PASS: mars
    restart: unless-stopped
```

RabbitMQ

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	actuator.queue.actuator.trigger.events	classic	Args	running	0	0	0			
/	dashboard.queue.actuator.trigger.events	classic	Args	running	0	0	0			
/	dashboard.queue.sensor.rest.#.events	classic	Args	running	0	0	0	1.6/s	1.6/s	1.6/s
/	dashboard.queue.sensor.telemetry.#.events	classic	Args	running	0	1	1	1.4/s	1.4/s	1.4/s
/	rules.queue.rules.#.events	classic	Args	running	0	0	0			
/	rules.queue.sensor.rest.#.events	classic	Args	running	0	0	0	1.6/s	1.6/s	1.6/s
/	rules.queue.sensor.telemetry.#.events	classic	Args	running	0	1	1	1.4/s	1.4/s	1.4/s

Rule Service

Postgres

The Rule Service consumes all sensor events, loads active rules from Postgres and evaluates conditions (>, <, >=, <=, =) against the measured value.

- rule_engine.py
- rules_repository.py
- rule_models.py
- db.py
- handlers.py

API Endpoints:

- GET /health
- GET /rules

DB Schema (rule)

id	SERIAL PRIMARY KEY
name	VARCHAR(100) NOT NULL
sensor_name	Identifies the source
metric_name	The metric field of the measurement
operator	> < >= <= == !=
treshold_value	FLOAT comparison treshold
unit	Optional, filters by unit
actuator_name	Name of the actuator to drive
target_state	State to send to the actuator
rule_enabled	BOOLEAN - defalut TRUE

DB Schema (rule)

python.py

```python
class RuleEvent(BaseModel):
    event_id: str
    operation: Literal["add", "toggle", "update", "delete"]
    rule_id: int | None = None
    rule_name: str | None = None
    rule_enabled: bool | None = None
    sensor_name: str | None = None
    metric_name: str | None = None
    actuator_name: str | None = None
    target_state: str | None = None
    threshold_value: float | None = None
    operator: str | None = None
    unit: str | None = None
```

Docker Infrastructure

Each service has its own Dockerfile, they are all orchestrated by the docker-compose.yml file.

- **Multi-Service Architecture:** Each component is isolated in its own Docker container.
- **Orchestration:** A single *docker-compose.yml* manages the entire stack, including the provided Mars IoT simulator and the message broker.
- **One-Step Deployment:** The environment is fully reproducible and starts with a single command: *docker compose up*.

Containers

Service	Image / Build	Host port	Depends on
mars-simulator	mars-iot-simulator:multiarch_v1	8080	–
mars-rabbitmq	rabbitmq:4.2.4-management	5672, 15672	–
mars-ingestion-service	build ./ingestion-service	8000	rabbitmq, simulator
mars-db	postgres:16	5432	simulator
mars-rule-service	build ./rule-service	8001	ingestion, rabbitmq, postgres (healthy)
mars-dashboard-service	build ./dashboard-service	8003	ingestion
mars-actuator-service	build ./actuator-service	8004	rule-service

Frontend - Sensors dashboard

Sensors

greenhouse_temperature
25.57 C
temperature_c (C)
status: ok
3/10/26, 2:30 PM

entrance_humidity
34.24 %
humidity_pct (%)
status: ok
3/10/26, 2:30 PM

co2_hall
729.56 ppm
co2_ppm (ppm)
status: ok
3/10/26, 2:30 PM

hydroponic_ph
6.51 pH
ph (pH)
status: ok
3/10/26, 2:30 PM

water_tank_level
70.13 %
level_pct (%)
status: ok
3/10/26, 2:30 PM

water_tank_level
2805.2 L
level_liters (L)
status: ok
3/10/26, 2:30 PM

corridor_pressure
100.44 kPa
pressure_kpa (kPa)
status: ok
3/10/26, 2:30 PM

air_quality_pm25
16.65 ug/m3
pm1 (ug/m3)
status: ok
3/10/26, 2:30 PM

air_quality_pm25
20.17 ug/m3
pm25 (ug/m3)
status: ok
3/10/26, 2:30 PM

air_quality_pm25
24.96 ug/m3
pm10 (ug/m3)
status: ok
3/10/26, 2:30 PM

air_quality_voc
211.68 ppb
voc_ppb (ppb)
status: ok
3/10/26, 2:30 PM

air_quality_voc
492.3 ppm
co2e_ppm (ppm)
status: ok
3/10/26, 2:30 PM

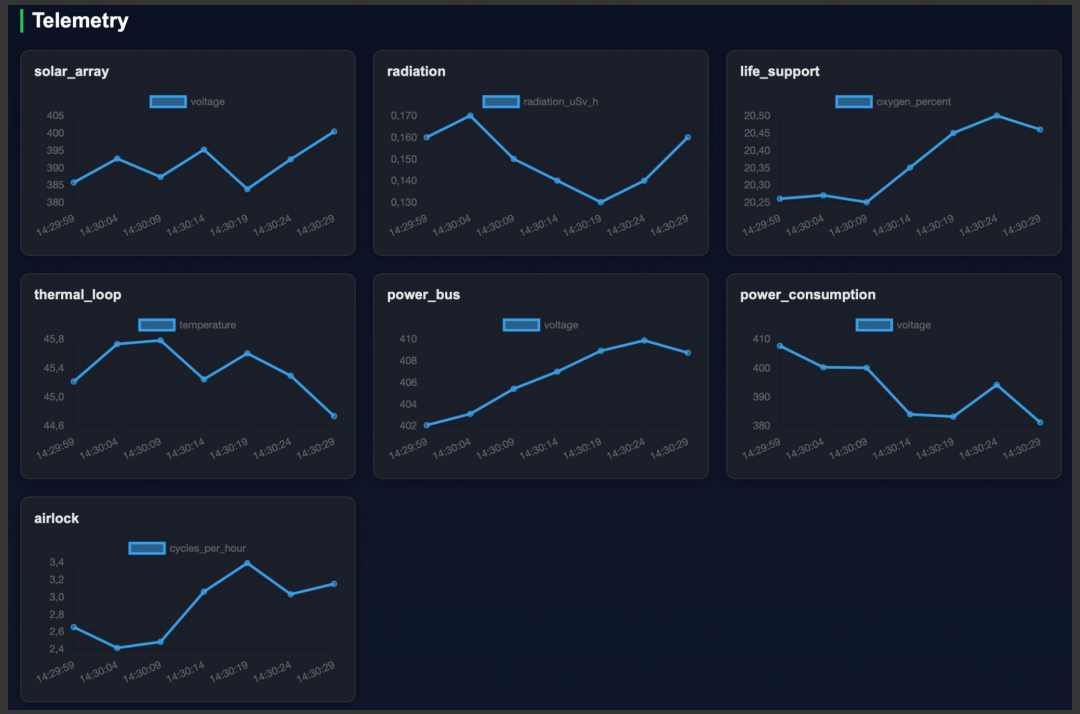
solar_array
387.3 V
voltage (V)
status: ok
3/10/26, 2:30 PM

solar_array
327.99 A
current (A)
status: ok
3/10/26, 2:30 PM

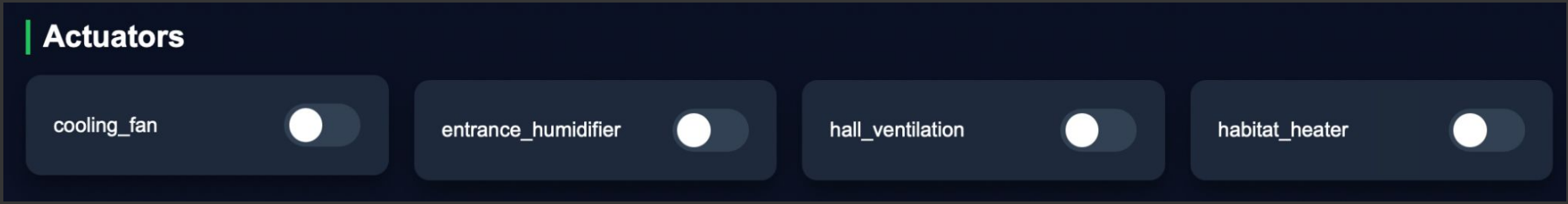
solar_array
127.03 kW
power (kW)
status: ok
3/10/26, 2:30 PM

solar_array
8422.244 kWh
cumulative_energy (kWh)
status: ok
3/10/26, 2:30 PM

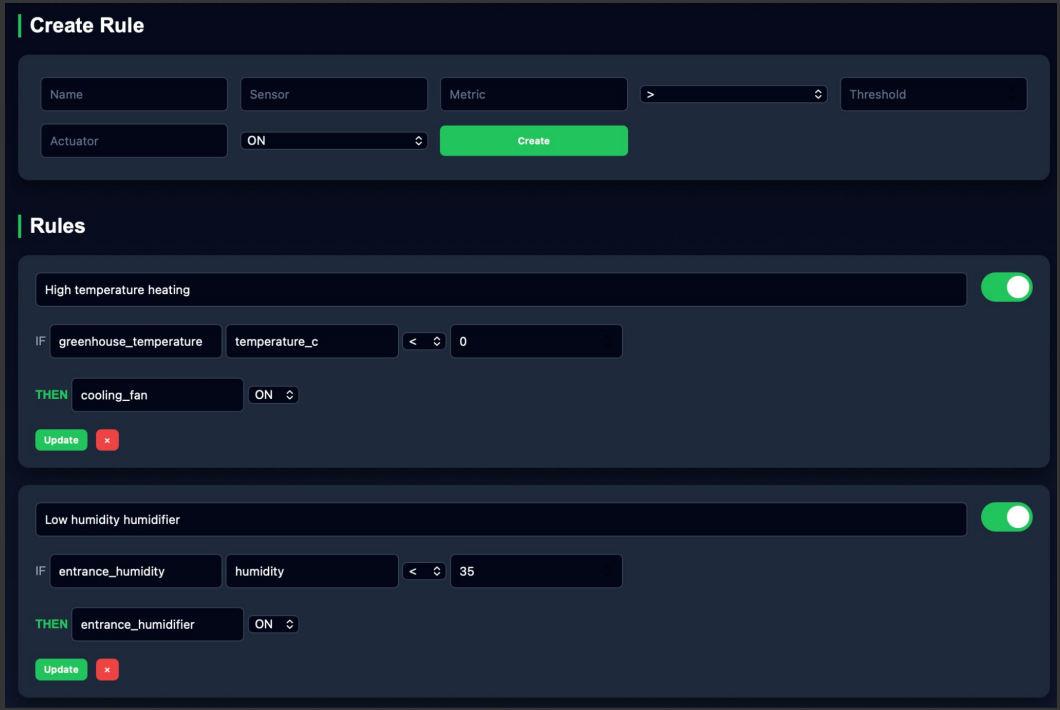
Frontend - Telemetry’s dashboard



Frontend - Actuators dashboard



Frontend - Rules Management



References

- **Repo:**
  - https://github.com/amedeobenedetti/2046211_MSC

Thank you for the attention!